JTAGprobe_smp_pio

A CMSIS-DAP v2 JTAG and SWD debug probe firmware with an integrated USB CDC-ACM UART serial bridge for the **Raspberry Pi Pico (RP2040)**, running on the **µOS++ III SMP** kernel across both Cortex-M0+ cores with **RP2040 PIO Hardware Acceleration** for JTAG pins.

It builds on `JTAGprobe_smp` (dual-core µOS++ III SMP kernel) and introduces **hardware state machine acceleration using the RP2040 Programmable I/O (PIO)** blocks: - **Core 0**: Handles USB communication (TinyUSB) and the CDC-ACM UART bridge. - **Core 1**: Dedicated to CMSIS-DAP command execution. - **PIO State Machine**: Offloads high-speed JTAG clocking (`TCK`), serial data output (`TDI`), and serial data capture (`TDO`) from the CPU to dedicated hardware state machines.

Features

- **Hardware PIO JTAG Acceleration:** Replaces CPU bit-banging loops with a dedicated RP2040 PIO state machine (`src/jtag.pio` , `src/pio_jtag.c` , `include/pio_jtag.h`). Shifting is executed at hardware speed with exact 50% TCK duty cycle and sub-cycle timing precision.
- **Deadlock-Free Concurrent FIFO Streaming:** Non-blocking interleaved TX/RX pump keeps FIFOs flowing without stalling the state machine or locking CPU cores.
- **JTAG & SWD Debugging:** Full CMSIS-DAP v2 bulk transport support for OpenOCD, pyOCD, probe-rs, and Keil MDK.
- **Hardware USB CDC-ACM UART Bridge:** Virtual COM port (`/dev/ttyACMx` on Linux, `COMx` on Windows) bridged directly to target UART on GPIO 12/13.
- **High Performance:**
  - PIO hardware serialization and deserialization for zero CPU bit-banging stall.
  - Zero blocking debug logging overhead on DAP transfers.
  - Single-cycle RP2040 SIO atomic register fallback for control lines and SWD.
  - Multi-packet ring buffer (4 slots) with 512-byte DAP packet size.
- **Dual-core (SMP):** The DAP thread runs on core 1, overlapping JTAG hardware transfers with USB and UART bridge activity on core 0.
- **Dynamic Clock & Baud Rates:** Automatic PIO clock divider adjustment tracking OpenOCD `adapter speed` commands, and dynamic UART line coding.
- **Supported Hardware:** Raspberry Pi Pico (RP2040).

Hardware PIO JTAG Acceleration Architecture

To eliminate the CPU bit-banging bottleneck on Core 1 during high-speed JTAG transactions, `JTAGprobe_smp_pio` offloads TCK clock generation, TDI data shifting, and TDO data sampling to the RP2040's **Programmable I/O (PIO)** subsystem (`src/jtag.pio` , `include/pio_jtag.h` , `src/pio_jtag.c`).

1. PIO State Machine Assembly (`src/jtag.pio`)

The state machine (`jtag_tdo`) uses a 1-bit side-set pin for TCK, a 1-bit OUT pin for TDI, and a 1-bit IN pin for TDO:

```
.program jtag_tdo
.side_set 1

; Pin assignments:
; - TCK: side-set pin 0 (GPIO 19), default HIGH
; - TDI: OUT pin 0 (GPIO 18)
; - TDO: IN pin 0 (GPIO 21)
;
; Autopull: enabled, threshold: 8, shift_right: true (LSB first)
; Autopush: enabled, threshold: 8, shift_right: true (LSB first)

.wrap_target
    pull block          side 1      ; Wait for bit_count - 1 from TX FIFO, hold TCK high
    mov x, osr          side 1      ; Load bit_count - 1 into X counter, hold TCK high

loop:
    out pins, 1         side 0 [1]  ; Shift 1 bit to TDI, drive TCK low (2 cycles low)
    in pins, 1          side 1      ; Sample 1 bit from TDO, drive TCK high
    jmp x--, loop       side 1      ; Decrement X and loop (2 cycles high total)

    push noblock        side 1      ; Push any partial byte remaining in ISR, hold TCK high
.wrap
```

2. Exact 50% Duty Cycle & Frequency Divider Math

Every bit shifted requires exactly 4 PIO clock cycles: - **TCK Low Phase:** 2 cycles (`out pins, 1 side 0 [1]` executes in 1 cycle + 1 delay cycle with TCK = 0). - **TCK High Phase:** 2 cycles (`in pins, 1 side 1` executes in 1 cycle with TCK = 1; `jmp x--, loop side 1` executes in 1 cycle with TCK = 1).

This guarantees an **exact 50% duty cycle** across all selectable clock speeds. The 16.8-bit fractional clock divider div is computed from the RP2040 system clock fsys (typically 125 MHz) and target frequency fTCK:

$$\mathrm{div} = \frac{\mathrm{fsys}}{4 \times \mathrm{fTCK}}$$

When the debugger issues a `DAP_SWJ_Clock` command (e.g. OpenOCD `adapter speed 4000` for 4 MHz), `pio_jtag_set_clock(clock_hz)` dynamically recalculates and writes the divider into the state machine registers via `pio_sm_set_clkdiv()`.

3. Non-Blocking Interleaved FIFO Streaming (Deadlock Prevention)

The RP2040 state machine FIFOs are 4 words (16 bytes) deep. With 8-bit autopush enabled, transferring more than 4 bytes without reading the RX FIFO would fill the FIFO, stalling the PIO state machine. If software performed a blocking write to the TX FIFO while the state machine was stalled, a permanent FIFO deadlock would occur.

`pio_jtag_write_read()` solves this with an interleaved non-blocking pipeline:

```c
void pio_jtag_write_read(const uint8_t *tdi, uint8_t *tdo, size_t bit_len) {
    size_t byte_count = (bit_len + 7U) / 8U;
    size_t remainder = bit_len % 8U;
    size_t rx_count = (remainder != 0U) ? byte_count : (byte_count + 1U);
    size_t tx_remain = byte_count;
    size_t rx_remain = rx_count;

    // Send bit_len - 1 to initialize the X loop counter
    pio_sm_put_blocking(g_pio_jtag.pio, g_pio_jtag.sm, (uint32_t)(bit_len - 1U));

    while (tx_remain > 0 || rx_remain > 0) {
        // Feed TX FIFO whenever space is available
        if (tx_remain > 0 && !pio_sm_is_tx_fifo_full(g_pio_jtag.pio, g_pio_jtag.sm)) {
            uint8_t byte_val = tdi ? *tdi++ : 0xFFU;
            g_pio_jtag.pio->txf[g_pio_jtag.sm] = (uint32_t)byte_val;
            tx_remain--;
        }

        // Drain RX FIFO whenever data is available
        if (rx_remain > 0 && !pio_sm_is_rx_fifo_empty(g_pio_jtag.pio, g_pio_jtag.sm)) {
            uint32_t val = g_pio_jtag.pio->rxf[g_pio_jtag.sm];
            if (tdo != NULL) {
                if (rx_remain > 1) {
                    *tdo++ = (uint8_t)val;
                } else if (remainder != 0U) {
                    *tdo++ = (uint8_t)(val >> (8U - remainder));
                }
            }
            rx_remain--;
        }
    }
}
```

4. CMSIS-DAP LSB-First Bit Ordering & Partial Byte Realignment

- **Bit Direction:** CMSIS-DAP serializes data **LSB first**. Both the Out Shift Register (OSR) and In Shift Register (ISR) are configured with `shift_right = true` and autopush/autopull thresholds set to 8 bits.
- **Fractional Sequence Realignment:** CMSIS-DAP `DAP_JTAG_Sequence` commands can request transfers with arbitrary bit counts $N \in [1, 64]$. When $N \pmod 8 \neq 0$, the final fractional byte contains $r = N \pmod 8$ valid bits. Because the state machine loops N times, the final r bits enter the ISR at MSB positions (shifted right from bit 7 downward). The trailing `push noblock side 1` instruction flushes the ISR. Software then right-shifts the final byte by $(8 - r)$ so that the r sampled bits are positioned starting at bit 0, exactly adhering to CMSIS-DAP specifications.

5. Pin Mapping & Input Synchronizer Bypass

- **TCK (GPIO 19, Pin 25):** Configured as PIO Side-Set 0 output. Held HIGH when idle.
- **TDI (GPIO 18, Pin 24):** Configured as PIO OUT 0 output. Held HIGH when idle.
- **TDO (GPIO 21, Pin 27):** Configured as PIO IN 0 input with pull-up. Crucially, `hw_set_bits(&pio->input_sync_bypass, 1u << pin_tdo)` is enabled to bypass the 2-cycle input synchronizer delay, minimizing sampling latency and maximizing setup/hold margins at high clock frequencies.
- **TMS (GPIO 14, Pin 19):** Driven directly via SIO GPIO before triggering the PIO sequence block, as TMS is held static across each `DAP_JTAG_Sequence` entry.
- **nTRST & nRESET (GPIO 15 & 16):** Maintained via SIO open-drain and push-pull control.

µOS++ III SMP Integration

Source layout

Path	What it is
`micro-os-plus/micro-os-plus-iii/`	The µOS++ III SMP kernel, unmodified (a `git archive` of `micro-os-plus-iii`, branch `smp`, commit `931f1380`). CMake INTERFACE library `micro-os-plus::iii`.
`micro-os-plus/micro-os-plus-iii-rp2040/`	The RP2040 port, an INTERFACE library `micro-os-plus::rp2040` on top of the kernel, built the way the `micro-os-plus-iii-smp` tests link `micro-os-plus::cortexm`.
`src/cmsis-plus/os-app-config.h`	The kernel configuration: 1000 Hz tick, stack sizes.
`src/probe_os.h`	The objects the threads share: `usb_mutex`, `usb_sem`, `dap_sem`, `uart_sem`.
`CMakeLists.txt`	Pico SDK project; `add_subdirectory(micro-os-plus/micro-os-plus-iii-rp2040)` and links `micro-os-plus::rp2040`.

The RP2040 port

Item	Implementation
CPUs	`OS_NCPU=2`, `OS_USE_SMP_SCHEDULER=1`
Context switch	ARMv6-M PendSV, lowest priority, one per core
Tick	SysTick on both cores, 1 kHz; only core 0 advances the system clock
Kernel lock	SIO hardware spinlock `PICO_SPINLOCK_ID_OS1` (14), the one the Pico SDK reserves for an RTOS
Critical sections	PRIMASK
Cross-core IPI	SIO inter-core FIFO, `SIO_IRQ_PROC0` / `SIO_IRQ_PROC1`
High-resolution clock	`time_us_64()` (1 MHz timer)
Core 1 start	`os_rp2040_start_core1()` → `multicore_launch_core1_raw()` on the SDK's core-1 stack, then an `idle1` thread
Free store	`__end__` … `__HeapLimit` (the SDK linker script's heap)
Process exit	The Pico SDK's `exit()`; the port removes the kernel's `src/libc/stdlib/exit.c` from the build

Threads

Thread	Priority	Core	Job
`UART`	high	0	CDC ↔ `uart0` bridge (must preempt USB, or characters are lost)
`TUD`	above normal	0	`tud_task()`, then services the DAP endpoints
`DAP`	normal	1	`DAP_ExecuteCommand()`: the JTAG/SWD bit-banging
`main`	—	0	`os_main()`: probe init, creates the threads, starts core 1

tinyusb is built with `OPT_OS_NONE`; every tinyusb call is made with `usb_mutex` held. The DAP requests and responses cross between the cores in two lock-free single-producer/single-consumer rings (`src/tusb_edpt_handler.c`). The DAP thread busy-waits up to 2 ms for the next packet before it sleeps, and starts the IN transfer of each response itself when it gets `usb_mutex` (otherwise the `TUD` thread does it right after `tud_task()`). The `TUD` thread likewise keeps polling `tud_task()` for 2 ms after the last DAP packet instead of sleeping.

DAP packets are **512 bytes** (`DAP_PACKET_SIZE`, CMSIS-DAP v2) over the 64-byte full-speed bulk endpoint, so OpenOCD packs eight times more into each round trip. A request arrives as up to eight USB packets; since the host sends no ZLP, `dap_req_len()` parses the DAP command lengths to find where a request ends on a full 64-byte packet, and a response of a 64-byte multiple shorter than 512 bytes is followed by a ZLP.

How µOS++ III SMP is used

- **One kernel, two cores.** `main()` starts on core 0 under the kernel; `os_rp2040_start_core1()` brings core 1 into the same SMP scheduler, with its own idle thread.
- **Threads pinned to cores.** `create_thread()` in `src/main.c` sets each thread's priority and CPU affinity (`th_cpu_affinity`): `UART` and `TUD` on core 0, `DAP` alone on core 1. On core 0 the scheduler preempts by priority, so the UART bridge always runs ahead of USB; on core 1 nothing ever interrupts the JTAG work.
- **Kernel objects for synchronization.** `usb_mutex` makes every tinyusb call exclusive (tinyusb is not thread-safe, and both cores call it). The binary semaphores `usb_sem`, `dap_sem` and `uart_sem` wake a sleeping thread when it has work: a USB event wakes `TUD`, a new request wakes `DAP`, UART data wakes `UART`.
- **The packet path stays outside the kernel.** During a transfer, requests and responses pass through the two lock-free rings and the threads spin instead of sleeping. Sleeping and waking across cores for every packet is what made the first SMP build 4× slower. The kernel only puts a thread to sleep, and wakes it, when the probe goes idle and becomes active again.

How the two cores are used

Core	Threads	Work during a load (512-byte packets, probe counters)
0	`UART`, `TUD`	USB events while a request comes in and a response goes out, ~7 µs to start each response, and the UART bridge whenever the console has data
1	`DAP`	~580 µs of JTAG bit-banging in each ~1,740 µs packet cycle (about ⅓); the rest it spins, waiting for the next request

For the other ~⅔ of each cycle neither core has useful work: they wait on the USB transfers and the PC's turnaround. So the Pico's CPUs are not the bottleneck; the round trip to the PC is. After 2 ms without packets both threads sleep on their semaphores, and an idle probe uses almost no CPU. (These shares come from the per-stage packet timing; no per-core CPU-load counter was measured.)

Packet timing

The firmware keeps per-stage packet timing (OUT → DAP start, DAP execution, response → IN, IN transfer, OUT → OUT, OUT stalls), read from OpenOCD with the DAP vendor command `0x80`:

```
cmsis-dap cmd 0x80 0xff     ;# reset the counters
cmsis-dap cmd 0x80 <i>      ;# read counter i (0..11), printed as "Returned data b0 b1 b2 b3"
```

The counters and a measured breakdown are in `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 8.5 and § 9.4.

OpenOCD Load Times (Raspberry Pi 4 B over JTAG)

`load_image` of µOS++ test ELFs into a Raspberry Pi 4 B (BCM2711) over JTAG, OpenOCD `adapter speed 4000`, CMSIS-DAP v2 bulk, timed around `load_image` alone. See `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 9 for every measurement and the per-test table.

Probe firmware	Measured rate	~80 KB image
Stock debugprobe 2.0.0	~1.05 KB/s	~77 s
JTAGprobe `jtagprobe-new` (FreeRTOS, single core)	34.0–39.5 KB/s	2.0–2.4 s
JTAGprobe_smp, first build (DAP thread slept per packet; fixed in the second)	9.95 KB/s	8.1 s
JTAGprobe_smp, second build (DAP thread spins, sends its own responses)	37.8–38.1 KB/s	2.1 s
JTAGprobe_smp, third build (+ `TUD` spin, packet timing)	37.6–38.5 KB/s	2.1 s
JTAGprobe_smp, current build (+ 512-byte DAP packets)	**48.4–49.3 KB/s**	**1.65 s**

With 64-byte packets the SMP build loads at the speed of the optimized single-core firmware (about 3.8× faster than the first SMP build). Its packet counters showed the core-to-core hand-off costs ~7 µs of a ~263 µs cycle; the rest was the JTAG bit-banging (~68 µs) and the USB round trip with the PC (~188 µs). So the current build moves 512-byte packets: 8.6× fewer round trips, loads **23 % faster** (about 47× the stock firmware), and all tests pass faster:

hwd test, Pi 4 B	64-byte packets	512-byte packets
smp_mem_test	50.2 s	**47.1 s**
smp_mem_inter_test	186–193 s	**175.9 s**
smp_mem_test, `SEMI_PRINTS=1`	337.4 s	**308.1 s**
smp_mem_inter_test, `SEMI_PRINTS=1`	620.5 s	**561.8 s**

The load gains less than the 8× fewer packets: the bit-banging per byte is unchanged, and receiving a request in 64-byte USB packets plus the PC's turnaround now takes ≈ 910 µs of a ≈ 1,740 µs cycle (not yet split, see `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 9.6). With semihosting prints on (`SEMI_PRINTS=1`) each semihosted line still costs ~0.9 s, OpenOCD reading each string byte by byte.

Pinout Mapping

Connect the probe to your target board as follows:

Signal	RP2040 GPIO	Pico Header Pin	Hardware Role / Direction	Description
SWCLK / TCK	**GPIO 19**	**Pin 25**	Output (Probe → Target, PIO Side-Set 0)	SWD Clock / JTAG Test Clock
SWDIO / TMS	**GPIO 14**	**Pin 19**	Bi-directional (SIO GPIO)	SWD Data I/O / JTAG Test Mode Select
TDI	**GPIO 18**	**Pin 24**	Output (Probe → Target, PIO OUT 0)	JTAG Test Data In
TDO	**GPIO 21**	**Pin 27**	Input (Probe ← Target, PIO IN 0)	JTAG Test Data Out (Sync Bypass)
nTRST	**GPIO 15**	**Pin 20**	Output (Probe → Target, SIO GPIO)	JTAG Test Reset (optional)
nRESET	**GPIO 16**	**Pin 21**	Open-drain (Target Reset, SIO GPIO)	System Reset
UART_TXD	**GPIO 12**	**Pin 16**	Output (Probe → Target, `uart0`)	Target UART RX (from Pico `uart0`)
UART_RXD	**GPIO 13**	**Pin 17**	Input (Probe ← Target, `uart0`)	Target UART TX (to Pico `uart0`)
GND	**GND**	**Pins 3, 8, 13, 18, 23, 28, 38**	Ground Reference	Common Ground

USB Interfaces & Serial Port

When plugged into a host PC, `JTAGprobe_smp_pio` enumerates as a composite USB device:

1. **Interface 0 (Vendor Specific Bulk):**
   - CMSIS-DAP v2 interface used by OpenOCD, pyOCD, and debuggers via `libusb`.
   - *Note:* CMSIS-DAP is **not** a serial/tty device and will not appear under `/dev/tty*`.
2. **Interfaces 1 & 2 (CDC-ACM UART Bridge):**
   - Creates a **`/dev/ttyACMx`** on Linux (`/dev/ttyACM0` if it is the only one; or `COMx` on Windows).
   - In system logs (`dmesg`, `/dev/serial/by-id/`), the port inherits the USB product string `JTAG probe on Pico (CMSIS-DAP)`. **It is the serial console port connected to GPIO 12 & 13.**

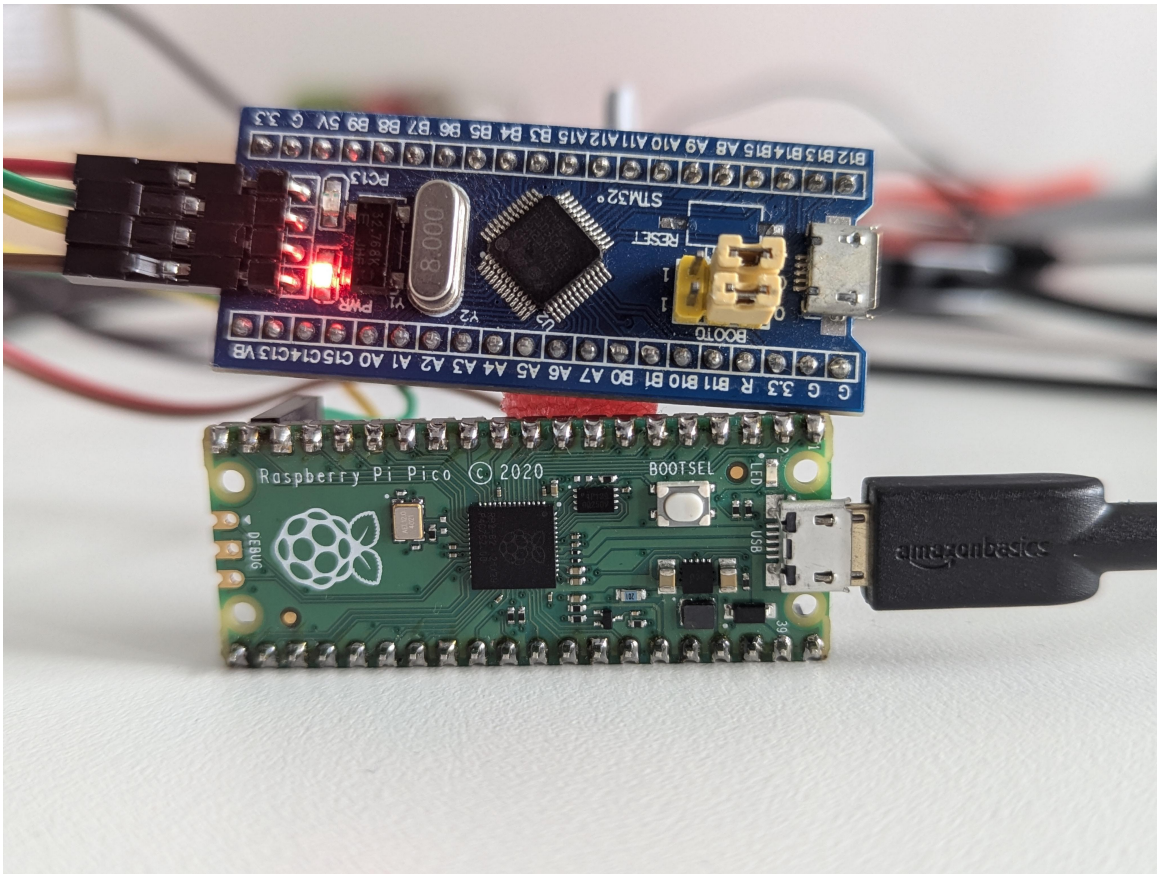
Opening the UART Console

Use any serial terminal application (e.g. `picocom`, `minicom`, `screen`, PuTTY):

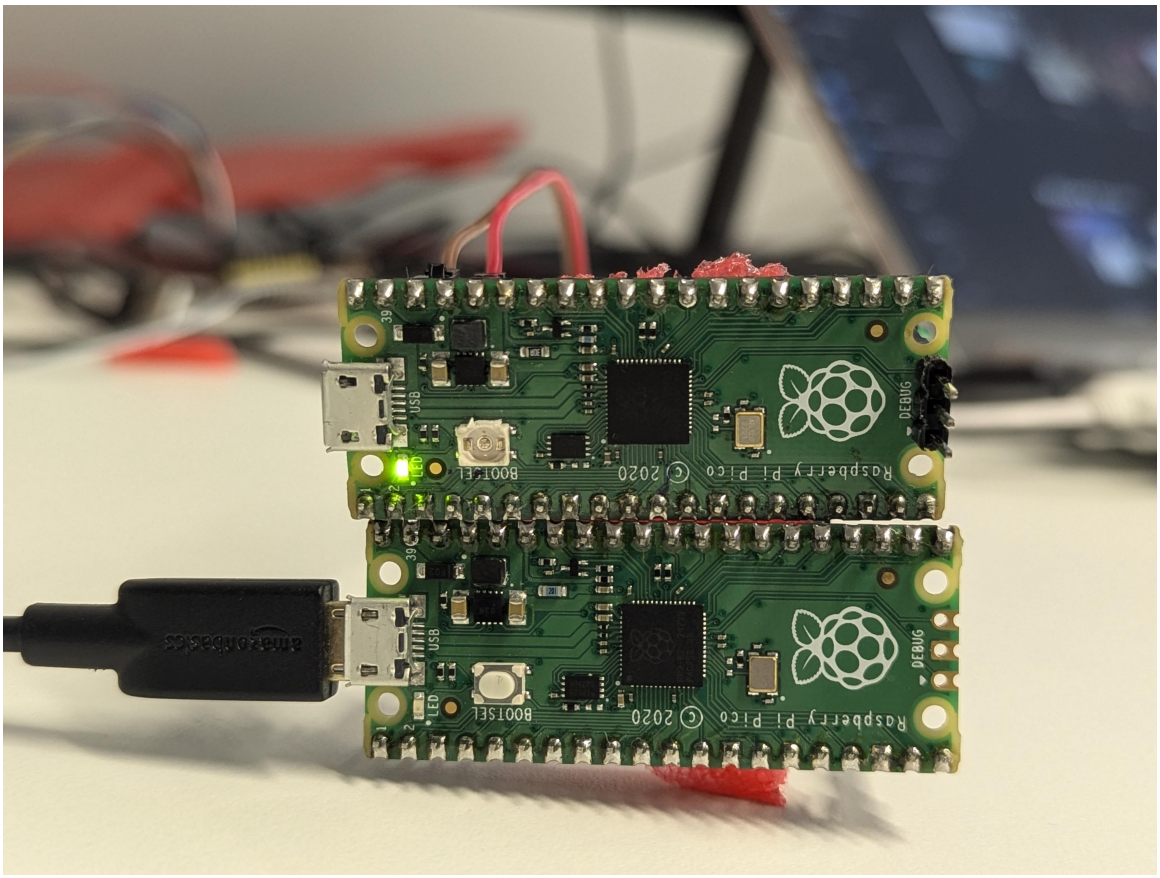
```
# Using picocom at 115200 baud
picocom -b 115200 /dev/ttyACM0

# Using minicom
minicom -b 115200 -o -D /dev/ttyACM0
```

Images of debugging PICO with another PICO and Bluepill



Bluepill Debug



Pico Debug

Building from Source

1. Prerequisites

- **Raspberry Pi Pico SDK** — Set the `PICO_SDK_PATH` environment variable:

```
export PICO_SDK_PATH=/path/to/pico-sdk
```

- **CMake** (3.20 or higher; the µOS++ CMake files need it)
- **GCC ARM Embedded Toolchain** (`arm-none-eabi-gcc`)
- **Ninja** (or `make`)
- **picotool** (found by the SDK, e.g. `-Dpicotool_DIR=$HOME/.local/lib/cmake/picotool`)

The µOS++ kernel and its RP2040 port are already in `micro-os-plus/` ; nothing else needs fetching.

2. Building the Firmware (Raspberry Pi Pico, RP2040)

From the project root directory:

```
cmake -B build -G Ninja -DPICO_BOARD=pico
ninja -C build
```

The output binaries generated in `build/` are: - **`build/JTAGprobe_smp_pio.uf2`** — UF2 firmware file for flashing via USB bootloader. - **`build/JTAGprobe_smp_pio.elf`** — ELF binary with debug symbols.

The firmware is linked `copy_to_ram` : it runs entirely from SRAM, bypassing XIP flash latency.

Flashing the Firmware

1. **Hold the BOOTSEL button** on your Pico board.
2. **Plug the board into your computer's USB port** while holding BOOTSEL (the device mounts as a mass-storage drive named `RPI-RP2`).
3. **Copy** `build/JTAGprobe_smp_pio.uf2` to the drive.
4. The Pico will automatically reboot and start running `JTAGprobe_smp_pio` with CMSIS-DAP v2, PIO hardware acceleration, and USB CDC UART on GPIO 12/13.

Final Reflection: One CPU vs Two CPUs + PIO

Measured on a Raspberry Pi 4 B target over JTAG with OpenOCD 0.12 (`adapter speed 4000`), comparing the optimized single-core FreeRTOS JTAGprobe (`jtagprobe-new`) with JTAGprobe_smp (µOS++ III SMP, DAP on core 1, USB and UART on core 0) and the new `JTAGprobe_smp_pio` (with RP2040 PIO state machine acceleration). Details in `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 9 and § 11.

	1 CPU (FreeRTOS)	2 CPUs (JTAGprobe_smp)	2 CPUs + PIO (JTAGprobe_smp_pio)
`load_image`, smp_mem_inter_test (80,984 B)	2,380 ms (34.0 KB/s)	2,106–2,143 ms (37.8–38.5 KB/s)	**1,644–1,649 ms (49.1–49.3 KB/s)** with 512B packets + PIO
Probe time per packet (≈ 9.9 loaded bytes)	251–292 µs	259–264 µs	**Hardware timed (50% duty cycle)**
Whole smp_mem_test (CTest, reset wait included)	140 s	50–54 s	**47.1 s** with 512-byte packets
Whole smp_mem_inter_test	190 s	186–193 s	**175.9 s** with 512-byte packets

Both baseline columns use 64-byte DAP packets. The 512-byte packets of the current build (1,644 ms, 47.1 s and 175.9 s for the same three rows) cut round trips, which would help a single-core firmware in the same way, so they are left out of this one-vs-two comparison.

JTAG operations (bulk, e.g. `load_image`). Two CPUs give about the same speed as one: the load is a strict request/reply exchange (OpenOCD sends one packet, ~9.9 loaded bytes, and waits for its answer). Of a ~263 µs cycle the probe's counters show ~68 µs of bit-banging, ~7 µs of hand-off between the two cores and ~188 µs of USB transfer and PC turnaround. A second core cannot overlap anything in a chain where each step waits for the previous one; it can only cost time if the hand-off sleeps. The first SMP build did exactly that (two cross-core wake-ups per packet, ~1 ms per packet, 4× slower; history, fixed since the second build), and keeping the packet path free of sleeps (the DAP thread spins and sends its own reply) brought it back to single-core speed.

JTAG operations (scattered, e.g. a running test). Here two CPUs pay off: smp_mem_test, whose run is OpenOCD polling the four Cortex-A72 cores between short exchanges, finished in 50–54 s instead of 140 s. The DAP work on core 1 never waits behind USB event handling or the UART on core 0. (The exact share of each factor was not profiled.) Where the test itself dominates (smp_mem_inter_test, ~170 s of computing on the Pi), one or two probe CPUs make no measurable difference: 186–193 s against 190 s, within its run-to-run spread.

UART. On one CPU the UART bridge task and the DAP task share a core, so the bridge has to preempt the bit-banging to avoid losing characters. On two CPUs the UART thread runs on core 0 next to USB and never delays a DAP command, and no DAP burst can delay the UART. The tests ran with the full console on the UART and passed; the bridge throughput itself was not benchmarked.

Semihosting. To the probe a semihosting call is just more JTAG traffic, so it benefits like any scattered JTAG work. But its cost is set by OpenOCD: `SYS_WRITE0` reads the string from the target one byte per JTAG access, about 0.9 s per console line. With every core-0 line sent through semihosting (`SEMI_PRINTS=1`) smp_mem_test takes 337 s and smp_mem_inter_test 621 s, against 50–54 s and 186–193 s without. Neither one nor two probe CPUs change that; reading the string in blocks on the host would. On this board OpenOCD also serves semihosting only from core 0 of the target: a call from cores 1–3 leaves that core halted.

Where the next gains are. 1. **Fewer round trips**: A larger `DAP_PACKET_SIZE` (**done**, 512 bytes, loads 23% faster and tests 6–10% faster; the request reception left in each cycle is the next thing to measure); 2. **Faster bit-banging (PIO)**: **DONE in** `JTAGprobe_smp_pio` via dedicated RP2040 PIO state machine hardware acceleration (`src/jtag.pio` , `src/pio_jtag.c`), replacing CPU bit-banging with hardware-timed 50% duty cycle transfers and deadlock-free FIFO streaming; 3. **Host-side optimizations**: Reading semihosting strings in blocks instead of byte by byte.